# PET

1. **Pattern Printing**

1

2 3

3 4 5

5 6 7 8

Sol.

```
#include <stdio.h>
int main()
{
   int n = 4;
   int start = 1;

   for (int i = 1; i <= n; i++)
   {
      int num = start;
      for (int j = 1; j <= i; j++)
      {
         printf("%d ", num);
         num++;
      }
      printf("\n");
      if(num == 2)
         start++;
      else
         start = num - 1;
   }
   return 0;
}
```

## 2. Wave Form of Matrix

i/p:- 3 3

1 2 3

4 5 6

7 8 9

o/p:-

1 4 7 8 5 2 3 6 9

```
#include <stdio.h>
int main()
{
   int r, c;
   printf("Enter rows and columns: ");
   scanf("%d %d", &r, &c);
   int a[r][c];
   printf("Enter matrix:\n");
   for(int i = 0; i < r; i++)
   {
      for(int j = 0; j < c; j++)
         scanf("%d", &a[i][j]);
   }
   printf("Wave output: ");
   for(int j = 0; j < c; j++)
   {
      if(j % 2 == 0)
      {
         for(int i = 0; i < r; i++)
            printf("%d ", a[i][j]);
      } else {
         for(int i = r - 1; i >= 0; i--)
            printf("%d ", a[i][j]);
      }
   }
}
```

3. **Print Binary & Reverse Binary → Decimal**

**Example**

Input: 10
Binary: 1010
Reverse: 0101 → Decimal = 5

Sol:-

```
#include <stdio.h>
int main() {
   int n, rev = 0, bin[32], i = 0;
   printf("Enter number: ");
   scanf("%d", &n);
   int temp = n;

   while(temp > 0) {
      bin[i++] = temp % 2;
      temp /= 2;
   }

   printf("Binary: ");
   for(int j = i-1; j >= 0; j--)
      printf("%d", bin[j]);

   printf("\nReverse: ");
   for(int j = 0; j < i; j++)
      printf("%d", bin[j]);

   for(int j = 0; j < i; j++)
      rev = rev * 2 + bin[j];

   printf("\nReverse Binary Decimal = %d\n", rev);

   return 0;
}
```

4. **Print N Bits from Given Position**
**Example**
Number: 23 → (10111)
Position = 4, N = 3 → Output: 101

Sol:-

```
#include <stdio.h>

int main()
{
   int num, pos, n;
   printf("Enter number: ");
   scanf("%d", &num);
   printf("Enter position and number of bits: ");
   scanf("%d %d", &pos, &n);

   int mask = ((1 << n) - 1) << (pos - n + 1);
   int bits = (num & mask) >> (pos - n + 1);

   printf("Extracted bits = %b\n", bits);
   return 0;
}
```

5. **Structure — Bank Account System (Mini Project)**
**Features**
✓ Create account
✓ Deposit
✓ Withdrawal
✓ Transaction history

Sol:-

```
#include <stdio.h>
#include <string.h>

struct Bank
{
   char name[50];
   long acc;
   int balance;
   int history[100];
```

```
    int hcount;
};

int main()
{
    struct Bank b;
    printf("Enter name: ");
    scanf("%s", b.name);
    printf("Enter account number: ");
    scanf("%ld", &b.acc);

    b.balance = 0;
    b.hcount = 0;

    int choice, amt;

    while(1)
    {
        printf("\n1.Deposit 2.Withdraw 3.History 4.Exit\n");
        scanf("%d", &choice);

        if(choice == 1)
        {
            printf("Enter amount: ");
            scanf("%d", &amt);
            b.balance += amt;
            b.history[b.hcount++] = amt;
        }
        else if(choice == 2)
        {
            printf("Enter amount: ");
            scanf("%d", &amt);
            if(amt <= b.balance)
            {
                b.balance -= amt;
                b.history[b.hcount++] = -amt;
            }
            else
            {
                printf("Insufficient balance\n");
            }
        }
        else if(choice == 3)
        {
            printf("Transaction history:\n");
            for(int i=0;i<b.hcount;i++)
                printf("%d\n", b.history[i]);
            printf("Current Balance = %d\n", b.balance);
        }
        else break;
    }
    return 0;
}
```

6. **Array Equilibrium Index**

Equilibrium index → left sum = right sum

**Example**

Input: [-7, 1, 5, 2, -4, 3, 0]

Output: 3 (because left sum = right sum = 0)

Sol:-

```
#include <stdio.h>

int main()
{
   int n;
   scanf("%d", &n);
   int a[n];

   for(int i = 0; i < n; i++)
      scanf("%d", &a[i]);

   int total = 0, left = 0;

   for(int i = 0; i < n; i++)
      total += a[i];

   for(int i = 0; i < n; i++)
   {
      total -= a[i];
      if(left == total)
      {
         printf("Equilibrium index = %d\n", i);
         return 0;
      }
      left += a[i];
   }

   printf("No equilibrium index\n");
   return 0;
}
```

7. **Replace Substring in Main String**
**Example**
Main string: "abcabc"
Find: "abc"
Replace: "xyz"
Output: "xyzxyz"

Sol:-

```
#include <stdio.h>
#include <string.h>

int main()
{
   char mainstr[200], find[50], rep[50], result[300];
   int i = 0, r = 0;

   printf("Enter main string: ");
   scanf(" %[^\n]", mainstr);

   printf("Enter substring to replace: ");
   scanf(" %[^\n]", find);

   printf("Enter new substring: ");
   scanf(" %[^\n]", rep);

   int findLen = strlen(find);

   while (mainstr[i] != '\0')
   {

      if (strncmp(&mainstr[i], find, findLen) == 0)
      {
         for (int k = 0; rep[k] != '\0'; k++)
            result[r++] = rep[k];
         i += findLen;
      }
      else
      {
         result[r++] = mainstr[i++];
      }
   }

   result[r] = '\0';

   printf("Result: %s\n", result);
   return 0;
}
```

8. **Reverse the number and convert the number to binary using recursion**

Enter a positive integer: 31
Reversed number: 13
Binary: 1101

**Sol:-**

```
#include <stdio.h>
// Reverse number using recursion
long reverse_num_rec(long n, long *rev)
{
   if (n == 0)
   {
      return 0;
   }
   *rev = *rev * 10 + (n % 10);
   reverse_num_rec(n / 10, rev);
//    return *rev;
}
// Print binary using recursion (MSB first)
void print_binary_rec(unsigned int n)
{
   if (n == 0)
   {
      return;
   }
   print_binary_rec(n / 2);
   printf("%d", n%2);
}

int main()
{
   long n;
   printf("Enter a positive integer: ");
   if (scanf("%ld", &n) != 1 || n < 0)
      return 0;

   // Reverse
   long rev = 0;
   if (n == 0)
      rev = 0;
   else
      reverse_num_rec(n, &rev);
   printf("Reversed number: %ld\n", rev);

   //binary
   if (rev == 0)
      printf("Binary: 0\n");
   else
   {
      printf("Binary: ");
      print_binary_rec((unsigned int)rev);
      printf("\n");
   }
}
```

9. **Print palindrome prime numbers up to N**

Print palindrome primes up to: 100
Palindrome primes <= 100:
2 3 5 7 11

Sol:-

```
#include <stdio.h>

// check palindrome
int is_palindrome(int x)
{
   int rev = 0, tmp = x;
   while (tmp > 0)
   {
      rev = rev*10 + (tmp % 10);
      tmp /= 10;

   }
   return rev == x;
}

int is_prime(int x)
{
   if (x < 2)
      return 0;
   if (x == 2)
      return 1;
   if (x % 2 == 0)
      return 0;
   for (int i = 3; i*i <= x; i += 2)
   {
      if (x % i == 0)
         return 0;
   }
   return 1;
}

int main()
{
   int n;
   printf("Print palindrome primes up to: ");
   scanf("%d", &n);
   printf("Palindrome primes <= %d:\n", n);
   for (int i = 2; i <= n; i++)
   {
      if (is_prime(i) && is_palindrome(i))
         printf("%d ", i);
   }
   printf("\n");
   return 0;
}
```

**10. Pascal's Triangle**

Enter number of rows: 5
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1

Sol:-

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int n;
   printf("Enter number of rows: ");
   scanf("%d", &n);
   for (int i = 0; i < n; i++)
   {
      for(int k = i; k < n; k++)
         printf(" ");

      long long val = 1;
      for (int j = 0; j <= i; j++)
      {
         if (j == 0)
            val = 1;
         else
            val = val * (i - j + 1) / j;
         printf("%lld ", val);
      }
      printf("\n");
   }
   return 0;
}
```

**11. Interleave words from two strings using Dynamic Memory Allocation (DMA)**

Enter first string: sai
Enter second string: uma
Interleaved result: suamia

Sol:-

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
   char buf1[201], buf2[201];
   printf("Enter first string: ");
   scanf(" %[^\n]", buf1);
   printf("Enter second string: ");
   scanf(" %[^\n]", buf2);

   int n1 = strlen(buf1), n2 = strlen(buf2);
   int outlen = n1 + n2;
   char *out = (char *)malloc(outlen + 1);
   if (!out) return 1;

   int i = 0, p = 0;
   while (i < n1 || i < n2)
   {
      if (i < n1) out[p++] = buf1[i];
      if (i < n2) out[p++] = buf2[i];
      i++;
   }
   out[p] = '\0';

   printf("Interleaved result: %s\n", out);
   free(out);
   return 0;
}
```

**12. Time convert to words**

Enter time in HH MM (0-23 0-59): 5 30
five thirty

sol:-

```
#include <stdio.h>

const char *words[] = {
   "zero","one","two","three","four","five","six","seven","eight","nine",
   "ten","eleven","twelve","thirteen","fourteen","fifteen","sixteen",
   "seventeen","eighteen","nineteen","twenty","twenty one","twenty two",
   "twenty three","twenty four","twenty five","twenty six","twenty seven",
   "twenty eight","twenty nine","thirty","thirty one","thirty two",
   "thirty three","thirty four","thirty five","thirty six","thirty seven",
   "thirty eight","thirty nine","forty","forty one","forty two",
   "forty three","forty four","forty five","forty six","forty seven",
   "forty eight","forty nine","fifty","fifty one","fifty two","fifty three",
   "fifty four","fifty five","fifty six","fifty seven","fifty eight","fifty nine"
};

int main()
{
   int hh, mm;
   printf("Enter time in HH MM (0-23 0-59): ");
   if (scanf("%d %d", &hh, &mm) != 2)
      return 0;
   if (hh < 0 || hh > 23 || mm < 0 || mm > 59)
   {
      printf("Invalid time\n");
      return 0;
   }

   int hour12 = hh % 12;
   if (hour12 == 0) hour12 = 12;

   if (mm == 0)
   {
      printf("%s o'clock", words[hour12]);
      if (hh == 0)
         printf(" midnight");
      else if (hh == 12)
         printf(" noon");
      printf("\n");
   }
   else
   {
      printf("%s %s\n", words[hour12], words[mm]);
   }
   return 0;
}
```

**13. 2D array 90° rotation (clockwise) — using DMA**

Enter rows and columns: 3 3
Enter matrix:
1 2 3
4 5 6
7 8 9
Rotated matrix (90 degrees clockwise):
7 4 1
8 5 2
9 6 3

Sol:-

```
#include<stdio.h>
#include<stdlib.h>

int main()
{
	int row;
	printf("Enter row : ");
	scanf("%d",&row);

	int col;
	printf("Enter col : ");
	scanf("%d",&col);

	int **arr = malloc(row * sizeof(int *));
	for (int i = 0; i < row; i++)
	   arr[i] = malloc(col * sizeof(int));

	printf("Enter arr elements :\n");
	for(int i=0;i<row;i++)
	{
		for(int j=0;j<col;j++)
		{
			scanf("%d",&arr[i][j]);
		}
	}

	printf("After rotate :\n");
	for(int i=0;i<col;i++)
	{
		for(int j=row-1;j>=0;j--)
		{
			printf("%d ",arr[j][i]);
		}
		printf("\n");
	}
	for (int i = 0; i < row; i++)
	   free(arr[i]);
	free(arr);
}
```

**14. Functions: frequency, multiplication, product using DMA**

Enter size: 4
Enter array A:
1 2 2 3
Enter array B:
4 5 6 7
Frequency of A:
Element 1 occurs 1 times
Element 2 occurs 2 times
Element 3 occurs 1 times

Element-wise product:
4 10 12 21

Dot product = 47

Sol:-

```
#include <stdio.h>
#include <stdlib.h>

void frequency(int *arr, int n)
{
   // naive approach: dynamic array for visited flags
   int *vis = calloc(n, sizeof(int));
   for (int i = 0; i < n; i++)
   {
      if (vis[i])
         continue;
      int count = 1;
      for (int j = i+1; j < n; j++)
      {
         if (arr[i] == arr[j])
         {
            count++;
            vis[j] = 1;

         }
      }
      printf("Element %d occurs %d times\n", arr[i], count);
   }
   free(vis);
}

int* elementwise_multiply(int *a, int *b, int n)
{
   int *res = malloc(n * sizeof(int));
   for (int i = 0; i < n; i++)
      res[i] = a[i] * b[i];
   return res;
}

long long dot_product(int *a, int *b, int n)
{
   long long sum = 0;
```

```
    for (int i = 0; i < n; i++)
        sum += (long long)a[i] * b[i];
    return sum;
}

int main()
{
    int n;
    printf("Enter size: ");
    scanf("%d", &n);
    int *a = malloc(n * sizeof(int));
    int *b = malloc(n * sizeof(int));

    printf("Enter array A:\n");
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    printf("Enter array B:\n");
    for (int i = 0; i < n; i++) scanf("%d", &b[i]);

    printf("\nFrequency of A:\n");
    frequency(a, n);

    int *elem_prod = elementwise_multiply(a, b, n);
    printf("\nElement-wise product:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", elem_prod[i]);
    printf("\n");

    long long dp = dot_product(a, b, n);
    printf("\nDot product = %lld\n", dp);

    free(a);
    free(b);
    free(elem_prod);
    return 0;
}
```

**15. Disk Using Recursion – Tower of Hanoi in C**

For n = 3
Move disk 1 from A → C
Move disk 2 from A → B
Move disk 1 from C → B
Move disk 3 from A → C
Move disk 1 from B → A
Move disk 2 from B → C
Move disk 1 from A → C

**Sol :-**

```
#include <stdio.h>

void towerOfHanoi(int n, char src, char aux, char dest)
{
   // Base case
   if (n == 1)
   {
      printf("Move disk 1 from %c → %c\n", src, dest);
      return;
   }

   // Move n-1 disks from src to aux
   towerOfHanoi(n - 1, src, dest, aux);

   // Move remaining disk from src to dest
   printf("Move disk %d from %c → %c\n", n, src, dest);

   // Move n-1 disks from aux to dest
   towerOfHanoi(n - 1, aux, src, dest);
}

int main()
{
   int n;

   printf("Enter number of disks: ");
   scanf("%d", &n);

   towerOfHanoi(n, 'A', 'B', 'C');

   return 0;
}
```

**16. Largest Palindrome in Given Array**

Enter size: 7
121 345 44 989 100 22 67
Largest Palindrome = 989

Sol:-

```
#include <stdio.h>

int isPalindrome(int n)
{
   int rev = 0, temp = n;
   while (n)
   {
      rev = rev * 10 + n % 10;
      n /= 10;
   }
   return rev == temp;
}

int main()
{
   int n, i;
   printf("Enter size: ");
   scanf("%d", &n);

   int a[n];
   for (i = 0; i < n; i++)
      scanf("%d", &a[i]);

   int largest = a[0];

   for (i = 0; i < n; i++)
      if (isPalindrome(a[i]) && a[i] > largest)
         largest = a[i];

   printf("Largest Palindrome = %d\n", largest);
   return 0;
}
```

**17. List All Subsets (Power Set) 2^(n-1)**

Enter size: 3
1 2 3

{ }
{ 1 }
{ 2 }
{ 1 2 }
{ 3 }
{ 1 3 }
{ 2 3 }
{ 1 2 3 }

**Sol:-**

```
//2^(n-1)
#include <stdio.h>

int main()
{
   int n;
   printf("Enter number of elements: ");
   scanf("%d", &n);

   int arr[n];
   printf("Enter %d unique elements: ", n);
   for (int i = 0; i < n; i++)
      scanf("%d", &arr[i]);

   int total = 1 << n;  // same as 2^n

   printf("\nPower Set (excluding empty set):\n");

   for (int i = 0; i < total; i++)   // start from 1 to skip empty set
   {
      printf("{ ");
      for (int j = 0; j < n; j++)
      {
         if (i & (1 << j))  // check if jth bit is set
            printf("%d ", arr[j]);
      }
      printf("}\n");
   }

   return 0;
}
```

**18. Remove Duplicates Without Extra Array**

Enter n: 8
3 5 3 8 5 9 8 2

After removing duplicates: 3 5 8 9 2

Sol:-

```
#include <stdio.h>

int main()
{
   int n;

   printf("Enter n: ");
   scanf("%d", &n);

   int a[n];
   for (int i = 0; i < n; i++)
      scanf("%d", &a[i]);

   for (int i = 0; i < n; i++)
   {
      for (int j = i + 1; j < n; j++)
      {
         if (a[i] == a[j])
         {
            for (int k = j; k < n - 1; k++)
            {
               a[k] = a[k + 1];
            }
            n--;
            j--;
         }
      }
   }

   printf("After removing duplicates: ");
   for (int i = 0; i < n; i++)
      printf("%d ", a[i]);

   return 0;
}
```

**19. Reverse a Number (Bitwise Bit Reversal)**

5
Binary of 5 → 00000000000000000000000000000101
Reversed bits → 10100000000000000000000000000000

Reverse bit output = 2684354560

Sol:-

```
#include <stdio.h>

int reverseBits(unsigned int n)
{
   unsigned int rev = 0;
   for (int i = 0; i < 32; i++)
   {
      rev <<= 1;
      rev |= (n & 1);
      n >>= 1;
   }
   return rev;
}

int main()
{
   unsigned int n;
   scanf("%u", &n);
   printf("Reverse bit output = %u\n", reverseBits(n));
   return 0;
}
```

**20. If Zero Found in 2D Matrix → Make That Row & Column Zero**

3 3
1 2 3
4 0 6
7 8 9

1 0 3
0 0 0
7 0 9

Sol:-

```
#include <stdio.h>
int main()
{
   int r, c;
   scanf("%d %d", &r, &c);
   int a[r][c], row[r], col[c];

   for (int i = 0; i < r; i++)
      row[i] = 0;
   for (int j = 0; j < c; j++)
      col[j] = 0;

   for (int i = 0; i < r; i++)
   {
      for (int j = 0; j < c; j++)
      {
         scanf("%d", &a[i][j]);
         if (a[i][j] == 0)
         {
            row[i] = 1;
            col[j] = 1;
         }
      }
   }

   for (int i = 0; i < r; i++)
   {
      for (int j = 0; j < c; j++)
      {
         if (row[i] || col[j])
            a[i][j] = 0;
      }
   }

   for (int i = 0; i < r; i++)
   {
      for (int j = 0; j < c; j++)
      {
         printf("%d ", a[i][j]);
      }
      printf("\n");
   }
}
```

**21. Sort 2D Array Without Extra Array**

3 3
9 2 6
1 3 8
4 5 7

1 2 3
4 5 6
7 8 9

Sol:-

```
#include <stdio.h>

int main()
{
   int r, c;
   scanf("%d %d", &r, &c);

   int a[r][c];

   for (int i = 0; i < r; i++)
   {
      for (int j = 0; j < c; j++)
         scanf("%d", &a[i][j]);
   }

   int total = r * c;

   for (int i = 0; i < total - 1; i++)
   {
      for (int j = i + 1; j < total; j++)
      {
         int *p1 = &a[i / c][i % c];
         int *p2 = &a[j / c][j % c];

         if (*p1 > *p2)
         {
            int temp = *p1;
            *p1 = *p2;
            *p2 = temp;
         }
      }
   }

   for (int i = 0; i < r; i++)
   {
      for (int j = 0; j < c; j++)
         printf("%d ", a[i][j]);
      printf("\n");
   }

   return 0;
}
```

**22. Divide Even & Odd Elements Without Extra Array**

Enter n: 8
12 7 4 9 2 5 6 11

Even-Odd arranged array: 12 6 4 2 9 5 7 11

**Sol:-**

```
#include <stdio.h>

int main()
{
   int a[50], n, left = 0, right;

   scanf("%d", &n);
   right = n - 1;

   for (int i = 0; i < n; i++)
      scanf("%d", &a[i]);

   while (left < right)
   {
      while (a[left] % 2 == 0 && left < right)
         left++;

      while (a[right] % 2 != 0 && left < right)
         right--;

      if (left < right)
      {
         int temp = a[left];
         a[left] = a[right];
         a[right] = temp;
      }
   }

   printf("Even-Odd arranged array: ");
   for (int i = 0; i < n; i++)
      printf("%d ", a[i]);

   return 0;
}
```

**23. Car (Speed Control Simulation)**

Explanation:
Simulate a car with:
Start

Stop

Accelerate

Decrease speed

Sol:-

```
#include <stdio.h>

int main()
{
   int speed = 0, choice;

   do
   {
            printf("\n1. Start Car\n2. Stop Car\n3. Accelerate\n4. Decrease
      Speed\n5.Exit\n");
      scanf("%d", &choice);

      switch(choice)
      {
         case 1: speed = 10; printf("Car started. Speed = %d\n", speed); break;
         case 2: speed = 0; printf("Car stopped.\n"); break;
         case 3: speed += 10; printf("Speed = %d\n", speed); break;
         case 4:
            if(speed >= 10) speed -= 10;
            printf("Speed = %d\n", speed);
            break;
      }
   } while(choice != 5);

   return 0;
}
```

**24. Fahrenheit to Celsius (Start / Stop / Increment)**

Explanation:

Use a loop:
Start → End → Step
Convert each Fahrenheit to Celsius.

Sol:-

```
#include <stdio.h>

int main()
{
   int start, stop, step;
   printf("Enter start stop increment: ");
   scanf("%d %d %d", &start, &stop, &step);

   for (int f = start; f <= stop; f += step)
   {
      float c = (f - 32) * 5.0 / 9.0;
      printf("%d F = %.2f C\n", f, c);
   }
}
```

**25. Reverse a Word (String)**

**Sample:**
Input → hello
Output → olleh

Sol:-

```
#include <stdio.h>

int main()
{
   char s[100];
   scanf("%s", s);

   int l=0;
   while(s[l])
      l++;

   int i=0, j=l-1;
   while(i<j)
   {
      char t=s[i]; s[i]=s[j]; s[j]=t;
      i++; j--;
   }

   printf("%s", s);
}
```

**26. Prime numbers upto m and n**

5 20
5 7 11 13 17 19

Sol:-

```
#include <stdio.h>
int isPrime(int n)
{
   if(n<2)
      return 0;
   for(int i=2;i*i<=n;i++)
      if(n%i==0) return 0;
   return 1;
}
int main()
{
   int m,n;
   scanf("%d %d", &m,&n);

   for(int i=m;i<=n;i++)
   {
      if(isPrime(i))
         printf("%d ", i);
   }
}
```

**27. Duplicate array → return 0 else return 1**

n = 5
arr = 1 5 2 1 3
Duplicate found → 1

Sol:-

```
#include <stdio.h>
int main()
{
   int n;
   scanf("%d",&n);
   int a[n];
   for(int i=0;i<n;i++)
      scanf("%d",&a[i]);

   for(int i=0;i<n;i++)
   {
      for(int j=i+1;j<n;j++)
      {
         if(a[i]==a[j])
         {
            printf("0");
            return 0;
         }
      }
   }
   printf("1");
}
```

**28. Reverse the Vowels in a String**

**Hello**

**Holle**

**Sol:-**

```
#include <stdio.h>
#include <string.h>
int isvowel(char c)
{
   int x = 0;
   if(c == 'a'||c == 'e'||c == 'i'||c == 'o'||c == 'u')
      x = 1;
   return x;
}

int main()
{
   char s[100];
   scanf("%s", s);

   int i=0,j=strlen(s)-1;
   while(i<j)
   {
      if(!isvowel(s[i]))
         i++;
      else if(!isvowel(s[j]))
         j--;
      else
      {
         char t=s[i];
         s[i]=s[j];
         s[j]=t;
         i++;
         j--;
      }
   }
   printf("%s", s);
}
```

**29. Structure — find age**

```
#include <stdio.h>
struct Person
{
   char name[50];
   int birth;
};
int main()
{
   struct Person p;
   scanf("%s %d", p.name, &p.birth);
   printf("Age = %d", 2024 - p.birth);
}
```

**30. Insert into sorted list**

List = 1 2 3 5 6
New = 4
Output = 1 2 3 4 5 6

Sol:-

```
#include <stdio.h>
int main()
{
   int n,x;
   scanf("%d",&n);

   int a[n+1];
   for(int i=0;i<n;i++)
      scanf("%d",&a[i]);
   scanf("%d",&x);

   int i=n-1;
   while(i>=0 && a[i]>x)
   {
      a[i+1]=a[i];
      i--;
   }
   a[i+1]=x;

   for(int j=0;j<=n;j++)
      printf("%d ",a[j]);
}
```

**31. Palindrome numbers from 100–200**

101 111 121 131 141 151 161 171 181 191

Sol:-

```
#include <stdio.h>
int rev(int n)
{
   int r=0;
   while(n)
   {
      r=r*10+(n%10);
      n/=10;

   }
   return r;
}

int main()
{
   for(int i=100;i<=200;i++)
   {
      if(i==rev(i))
         printf("%d ",i);
   }
}
```

**32. Toeplitz Matrix**

What is a Toeplitz Matrix?
A matrix is Toeplitz if every diagonal from top-left to bottom-right has the same element.

Example:
1 2 3
4 1 2
5 4 1
Here,
Main diagonal = 1,1,1
Next diagonal = 2,2
Next diagonal = 3
Another diagonal = 4,4
Another = 5
All diagonals have same repeating values, so it is a Toeplitz Matrix.

Sol:-

```
#include <stdio.h>

int main()
{
   int r,c;
   scanf("%d %d",&r,&c);

   int a[r][c];
   for(int i=0;i<r;i++)
   {
      for(int j=0;j<c;j++)
         scanf("%d",&a[i][j]);
   }

   for(int i=1;i<r;i++)
   {
      for(int j=1;j<c;j++)
      {
         if(a[i][j] != a[i-1][j-1])
         {
            printf("Not Toeplitz");
            return 0;
         }
      }
   }

   printf("Toeplitz Matrix");
}
```

**33. Sort an array of 0s, 1s, and 2s without using any sorting algorithm**

✓ We can do this by counting how many 0s, 1s, and 2s are present.

✓ Then rewrite the array using these counts.

Steps

Traverse the array and count

how many 0s

how many 1s

how many 2s

Rewrite the array:

first put all 0s

then all 1s

finally all 2s

sol:-

```
#include <stdio.h>

int main()
{
   int size = 8, i;
   int count0 = 0, count1 = 0, count2 = 0;

   int a[size];

   printf("Enter array elements (only 0,1,2): ");
   for(i = 0; i < size; i++)
   {
      scanf("%d", &a[i]);
      if(a[i] == 0)
         count0++;
      else if(a[i] == 1)
         count1++;
      else if(a[i] == 2)
         count2++;
   }

   // Rewrite the array
   i = 0;
   while(count0--)
      a[i++] = 0;
   while(count1--)
      a[i++] = 1;
   while(count2--)
      a[i++] = 2;

   printf("Sorted Array: ");
   for(i = 0; i < size; i++)
      printf("%d ", a[i]);

   return 0;
}
```

**34. Find Pivot Index Where Left Sum = Right Sum**

arr = [1, 2, 3, 4, 6]
Left of index 3 → elements = 1, 2, 3 → sum = 6
Right of index 3 → elements = 6 → sum = 6
Left sum = Right sum
So pivot index = 3

First calculate total sum of array.
Maintain left sum = 0.
For each element:
right_sum = total_sum − left_sum − current_element
If left_sum == right_sum → pivot found
Otherwise add current element to left_sum and continue.

Sol:-

```
#include <stdio.h>

int main()
{
   int n, i;
   printf("Enter size of array: ");
   scanf("%d", &n);

   int arr[n];
   printf("Enter array elements: ");
   for(i = 0; i < n; i++)
      scanf("%d", &arr[i]);

   int totalSum = 0;
   for(i = 0; i < n; i++)
      totalSum += arr[i];

   int leftSum = 0;
   int pivot = -1;

   for(i = 0; i < n; i++)
   {
      int rightSum = totalSum - leftSum - arr[i];
      if(leftSum == rightSum)
      {
         pivot = i;
         break;
      }
      leftSum += arr[i];
   }

   if(pivot != -1)
      printf("Pivot index: %d\n", pivot);
   else
      printf("No pivot index found.\n");

   return 0;
}
```

**35. Check if a string can become a palindrome by deleting at most one character**

Example:
s = "abxba"
Remove 'x' → becomes "abba"
So YES, it is a palindrome.

Sol:-

```
#include <stdio.h>
#include <string.h>
// Function to check if a substring is palindrome
int isPalindrome(char *s, int left, int right)
{
   while (left < right)
   {
      if (s[left] != s[right])
         return 0;
      left++;
      right--;
   }
   return 1;
}
// Main checker function
int canFormPalindrome(char *s)
{
   int left = 0;
   int right = strlen(s) - 1;

   while (left < right)
   {
      if (s[left] == s[right])
      {
         left++;
         right--;
      }
      else
      {
         // Try removing one char (either left or right)
         return isPalindrome(s, left + 1, right) ||
               isPalindrome(s, left, right - 1);
      }
   }
   return 1; // Already palindrome
}
int main()
{
   char str[100];
   printf("Enter a string: ");
   scanf("%s", str);

   if (canFormPalindrome(str))
      printf("Yes, it is a palindrome\n");
   else
      printf("No, it is not a palindrome\n");
}
```

## 36. Calculate Number of Seconds in Given Years Using MACRO

1 day = 24 hours
1 hour = 60 minutes
1 minute= 60 seconds
1 day = 24 × 60 × 60 = 86400 seconds
365 days = 365 × 86400 seconds

Sol:-

```
#include <stdio.h>

#define SECONDS_IN_YEAR (365 * 24 * 60 * 60)

int main()
{
   int years;
   long long seconds;

   scanf("%d", &years);

   seconds = (long long) years * SECONDS_IN_YEAR;

   printf("Seconds in the year : %lld\n", seconds);

   return 0;
}
```

**37. Replace Specific Character Sequences in a String (bbc → $, bbe → &, bb → @)**

abbdebbch
a@de$h

sol:-

```
#include <stdio.h>

void replaceSequences(char *str)
{
   char result[200];
   int i = 0, j = 0;

   while (str[i] != '\0')
   {
      if (str[i] == 'b' && str[i+1] == 'b' && str[i+2] == 'c')
      {
         result[j++] = '$';
         i += 3;
      }
      else if (str[i] == 'b' && str[i+1] == 'b' && str[i+2] == 'e')
      {
         result[j++] = '&';
         i += 3;
      }
      else if (str[i] == 'b' && str[i+1] == 'b')
      {
         result[j++] = '@';
         i += 2;
      }
      else
      {
         result[j++] = str[i++];
      }
   }

   result[j] = '\0';

   // Copy back to original
   for (i = 0; result[i] != '\0'; i++)
      str[i] = result[i];
   str[i] = '\0';
}

int main()
{
   char str[200];

   scanf("%s", str);

   replaceSequences(str);

   printf("%s\n", str);
}
```

**38. Check if Two Strings Can Become Equal with at Most One Swap**

s1 = "bank"
s2 = "kanb"
b ↔ k
k ↔ b

Sol:-

```
#include <stdio.h>
#include <stdbool.h>
#include <string.h>

bool areAlmostEqual(char *s1, char *s2)
{
   int len = strlen(s1);
   int diff[2];
   int count = 0;

   for (int i = 0; i < len; i++)
   {
      if (s1[i] != s2[i])
      {
         if (count == 2)
            return false;

         diff[count++] = i;
      }
   }

   if (count == 0)
      return true;

   if (count == 2)
   {
      return (s1[diff[0]] == s2[diff[1]] &&
            s1[diff[1]] == s2[diff[0]]);
   }

   return false;
}

int main()
{
   char s1[100], s2[100];

   scanf("%s", s1);
   scanf("%s", s2);

   if (areAlmostEqual(s1, s2))
      printf("true\n");
   else
      printf("false\n");

   return 0;
}
```

**39. Find Number of Days in a Given Month**

12

The Number of days in the month 12 is 31.

Sol:-

```
#include <stdio.h>

int main()
{
   int month;
   scanf("%d", &month);

   switch(month)
   {
      case 1:
      case 3:
      case 5:
      case 7:
      case 8:
      case 10:
      case 12:
         printf("The Number of days in the month %d is 31.\n", month);
         break;

      case 4:
      case 6:
      case 9:
      case 11:
         printf("The Number of days in the month %d is 30.\n", month);
         break;

      case 2:
         printf("The Number of days in the February month is 28 or 29 (depending on
whether it's a leap year).\n");
         break;

      default:
         printf("Invalid month number. Please enter a number between 1 and 12.\n");
         break;
   }

   return 0;
}
```

**40. Transporting Boxes Through a Tunnel – Volume Validation Program**

Tunnel Rules:
Tunnel height = 41 feet
Tunnel width = infinite, so width doesn't matter.
A box has: length, width, height
A box can pass only if its height is strictly less than 41
(Height < 41)
4
5 5 5
1 2 40
10 5 41
7 2 42

125
80
failed
failed

sol:-

```
#include <stdio.h>

int main()
{
   int n;
   scanf("%d", &n);

   int l, w, h;

   while (n--)
   {
      scanf("%d %d %d", &l, &w, &h);

      // Check height limit
      if (h >= 41 || h > 40)
      {
         printf("failed\n");
      }
      else
      {
         int volume = l * w * h;
         printf("%d\n", volume);
      }
   }

   return 0;
}
```

**41. Insert a New Value into a Sorted Array**

Input: 1 4 6  New value = 5
Output: 1 4 5 6

Sol:-

```
#include <stdio.h>

int main()
{
   int size, arr[50], newVal, i, pos;

   scanf("%d", &size);

   for (i = 0; i < size; i++)
      scanf("%d", &arr[i]);

   scanf("%d", &newVal);

   // Find position to insert
   for (pos = 0; pos < size; pos++)
   {
      if (newVal < arr[pos])
         break;
   }

   // Shift elements to the right
   for (i = size; i > pos; i--)
   {
      arr[i] = arr[i - 1];
   }

   arr[pos] = newVal;
   size++;

   printf("Array elements after inserting : ");
   for (i = 0; i < size; i++)
      printf("%d ", arr[i]);

   return 0;
}
```

**42. Find the Smallest Positive Number Missing from an Unsorted Array**

Array: 5 5 2 1 4 -2
Present numbers: 1, 2, 4, 5
Missing: 3 → smallest missing positive = 3

Sol:-

```
#include <stdio.h>

int main()
{
   int size, arr[50], i, num = 1;
   scanf("%d", &size);

   for (i = 0; i < size; i++)
      scanf("%d", &arr[i]);

   while (1)
   {
      int found = 0;

      for (i = 0; i < size; i++)
      {
         if (arr[i] == num)
         {
            found = 1;
            break;
         }
      }

      if (!found)
      {
         printf("The smallest positive number missed is: %d", num);
         break;
      }

      num++; // check next positive number
   }

   return 0;
}
```

**43. GCD (HCF)**

Enter two numbers: 8 12
HCF= 4

```
#include <stdio.h>

int main()
{
   int a, b, min;

   printf("Enter two numbers: ");
   scanf("%d %d", &a, &b);

   min=a<b?a:b;

   while(min)
   {
      if(a%min==0 && b%min==0)
      {
         printf("HCF= %d\n",min);
         break;
      }
      min--;
   }


   return 0;
}
```

**44. Check mirror image of array or not**

1, 2, 3, 4
4, 3, 2, 1
Yes

Sol:-

```
#include <stdio.h>

int checkMirror(int n, int arr1[], int arr2[])
{
   for(int i = 0; i < n; i++)
   {
      if(arr1[i] != arr2[n - 1 - i])
      {
         return 1;   // not mirror
      }
   }
   return 0;   // mirror
}

int main()
{
   int arr1[4] = {1, 2, 3, 4};
   int arr2[4] = {4, 3, 2, 1};

   int res = checkMirror(4, arr1, arr2);

   if(res == 0)
      printf("Yes\n");
   else
      printf("No\n");

   return 0;
}
```

## 45. Case-Insensitive String Comparison Program

```
#include <stdio.h>
#include <ctype.h>

int areEqualIgnoreCase(char str1[], char str2[])
{
   int i = 0;

   while (str1[i] != '\0' && str2[i] != '\0')
   {
      if (tolower(str1[i]) != tolower(str2[i]))
      {
         return 0;   // not equal
      }
      i++;
   }

   // If both strings ended together → equal
   if (str1[i] == '\0' && str2[i] == '\0')
      return 1;

   return 0;   // lengths differ
}

int main()
{
   char str1[] = "hello";
   char str2[] = "Hello";

   if (areEqualIgnoreCase(str1, str2))
   {
      printf("The strings are equal.\n");
   }
   else
   {
      printf("The strings are not equal.\n");
   }

   return 0;
}
```

**46. Sort Cars by Year of Manufacture**

Enter the number of cars: 3

Enter details for Car 1:
Brand: Tata
Model: Vista
Year: 2014

Enter details for Car 2:
Brand: Kia
Model: Sonet
Year: 2013

Enter details for Car 3:
Brand: Kia
Model: Seltos
Year: 2015

Sorted Cars:
Brand: Kia, Model: Sonet, Year: 2013
Brand: Tata, Model: Vista, Year: 2014
Brand: Kia, Model: Seltos, Year: 2015

Sol:-

```
#include <stdio.h>

struct Car
{
   char brand[50];
   char model[50];
   int year;
};

int main()
{
   int n;
   printf("Enter the number of cars: ");
   scanf("%d", &n);

   struct Car cars[n];

   // Input details
   for (int i = 0; i < n; i++)
   {
      printf("\nEnter details for Car %d:\n", i + 1);
      printf("Brand: ");
      scanf("%s", cars[i].brand);
      printf("Model: ");
      scanf("%s", cars[i].model);
      printf("Year: ");
      scanf("%d", &cars[i].year);
   }

   // Sorting cars by year (ascending)
```

```
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            if (cars[i].year > cars[j].year)
            {
                struct Car temp = cars[i];
                cars[i] = cars[j];
                cars[j] = temp;
            }
        }
    }

    // Output sorted list
    printf("\nSorted Cars:\n");
    for (int i = 0; i < n; i++)
    {
        printf("Brand: %s, Model: %s, Year: %d\n",
               cars[i].brand, cars[i].model, cars[i].year);
    }

    return 0;
}
```

**47. Find the Missing Number in a Matrix and Print the Transpose**

```
3
1 2 3
4 0 6
7 8 9

9 * 10 / 2 = 45
1+2+3+4+0+6+7+8+9 = 40
45 – 40 = 5

Missing number is: 5
Transpose of corrected matrix:
1 4 7
2 5 8
3 6 9
```

Sol:-

```
#include <stdio.h>

int main()
{
    int n, i, j;
    int arr[10][10];
    int actualSum = 0, expectedSum, missing;
```

```
    scanf("%d", &n);

    // read matrix
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
            scanf("%d", &arr[i][j]);
    }

    // Calculate actual sum
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
            actualSum += arr[i][j];
    }

    // Expected sum of 1 to n*n
    expectedSum = (n * n * (n * n + 1)) / 2;

    // Missing number
    missing = expectedSum - actualSum;

    printf("Missing number is: %d\n", missing);

    // Replace the missing number (0)
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            if (arr[i][j] == 0)
                arr[i][j] = missing;
        }
    }

    // Print transpose
    printf("Transpose of corrected matrix:\n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
            printf("%d ", arr[j][i]);
        printf("\n");
    }

    return 0;
}
```

**48. Magic Number**

Example:
Number = 1729
Step 1:
1 + 7 + 2 + 9 = 19
Step 2:
1 + 9 = 10
Step 3:
1 + 0 = 1
Since final result = 1,
👉 1729 is a magic number

Sol:-

```
#include <stdio.h>

int main()
{
   int num, sum, temp;

   scanf("%d", &num);

   temp = num;

   // Repeat until sum becomes single digit
   while (temp > 9)
   {
      sum = 0;
      while (temp != 0)
      {
         sum += temp % 10;
         temp /= 10;
      }
      temp = sum;
   }

   if (temp == 1)
      printf("%d is a Magic Number", num);
   else
      printf("%d is not a Magic Number", num);

   return 0;
}
```